



Initial Pen Testing

Authors: Rich Macfarlane, Gordon Russell

An introduction to some Linux commands which could be useful for pentesting.

To reset all the check buttons from a previous attempt [click here](#)

Question 1 Using Locate

"locate" searches through the filesystem looking for things which match the command line. It does this very quickly using a file database.

Use the locate command to find the full path name of the nmap file with "stun-version" in its title.
file location:

Tests: Ready

Location search PASS

Locate finds all filesystem locations which contains information related to the search parameter nmap. Try

locate nmap

Pipe the output into grep to filter the output from the locate cmd, and find the location of an nmap script relating to "smb-flood".

file location:

Minimize

User: vinay mummadi (FULL) Account: 40807459@live.napier.ac.uk Registered

Account

Account Links

Machine State: RUN

Refresh in: 0:06.

Time left: Holding

Boot progress: complete

control

connect

stats

useful

Switch On

Nice Switch Off

Pull Out Power

Leave Queue

Kali 2024-4

☐ fresh linux install image

☒ use previous image (if available)

Check Tests: Ready

smb-flood location PASS

```
(kali@host-3-89)-[~]
$ locate stun-version
/usr/share/nmap/scripts/stun-version.nse

(kali@host-3-89)-[~]
$ locate nmap | grep smb-flood
/usr/share/nmap/scripts/smb-flood.nse

(kali@host-3-89)-[~]
$ locate bin/ | grep nmap
/usr/bin/nmap
/usr/bin/zenmap

(kali@host-3-89)-[~]
$ locate .db | grep locate
/var/lib/plocate/plocate.db

(kali@host-3-89)-[~]
$
```

Use locate and grep to find the full pathname where the nmap executable lives. This will be a directory with "bin/" somewhere in its name.

file location:

Check Tests: Ready

nmap binary location PASS

Use locate and grep to find the full pathname where the locate database lives. Hint: database files usually end with ".db".

file location:

Check Tests: Ready

locate db location PASS

Question 2 Network commands

Make sure you are logged into your virtual machine using at least 1 ssh session.

The ss command can be used to check network connections currently running. Use the ss command and identify the remote endpoint ip number used when you ssh to a virtual machine. Grep for ssh, or use a specific flag for the ss cmd to filter for tcp connections.

Proxy IP: 10.200.0.1

Check Tests: Ready

Proxy ssh IP PASS

The ss command with "-a" shows all current network connections. Services which are using ports will be in the LISTEN state.

How many ports are being used by services?

Service ports used:

48

Check Tests: Ready

Ports used PASS

Use the "sudo systemctl" command to start the apache2 service.

```
(kali@host-3-89)-[~]
$ ss -tn
State          Recv-Q      Send-Q      Local Address:Port      Peer Address:Port
ESTAB          0            0              10.0.3.89:22            10.200.0.1:41704

(kali@host-3-89)-[~]
$ ss -a | grep LISTEN | wc -l
48

(kali@host-3-89)-[~]
$ sudo systemctl start apache2

(kali@host-3-89)-[~]
$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Sat 2026-02-28 16:41:09 GMT; 12s ago
     Invocation: ab1b843a2e85434f9fe1e336a25bd2c2
       Docs: https://httpd.apache.org/docs/2.4/
   Process: 7084 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
    Main PID: 7100 (apache2)
      Tasks: 6 (limit: 9385)
     Memory: 20.4M (peak: 20.8M)
        CPU: 197ms
    CGroup: /system.slice/apache2.service
            └─7100 /usr/sbin/apache2 -k start
              7103 /usr/sbin/apache2 -k start
              7104 /usr/sbin/apache2 -k start
              7105 /usr/sbin/apache2 -k start
              7106 /usr/sbin/apache2 -k start
              7107 /usr/sbin/apache2 -k start

Feb 28 16:41:09 host-3-89.linuxzoo.net systemd[1]: Starting apache2.service - The Apache HTTP Server ...
Feb 28 16:41:09 host-3-89.linuxzoo.net systemd[1]: Started apache2.service - The Apache HTTP Server.
```

Check Tests: Ready

Start apache2 PASS

Check Tests: Ready

Start apache2 PASS

Use the `ss` command, find out which port number `apache2` is running on. Run `ss` using `sudo`, as this can then give you process information.

Hint. Use the man page/help of ss to check options to see the port numbers numerically, as well as to see what processes are using which entry in ss.

Service port: 80

```
(kali@host-3-89)-[~]
$ sudo ss -tnlp
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port
Process
LISTEN     0            32          192.168.1.254:53         0.0.0.0:*
users:((("dnsmasq",pid=927,fd=6)))
LISTEN     0            128         0.0.0.0:22               0.0.0.0:*
users:((("sshd",pid=826,fd=7)))
LISTEN     0            511         *:80                     *:80
users:((("apache2",pid=7107,fd=4),("apache2",pid=7106,fd=4),("apache2",pid=7105,fd=4),("apache2",pid=7104,fd=4),("apache2",pid=7103,fd=4),("apache2",pid=7100,fd=4)))
LISTEN     0            128         [::]:22                  [::]:*
users:((("sshd",pid=826,fd=8)))

(kali@host-3-89)-[~]
$ sudo ss -tnlp
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port
LISTEN     0            32          192.168.1.254:53         0.0.0.0:*
LISTEN     0            128         0.0.0.0:22               0.0.0.0:*
LISTEN     0            511         *:80                     *:80
LISTEN     0            128         [::]:22                  [::]:*

(kali@host-3-89)-[~]
$ sudo ss -tnlp | grep apache2
LISTEN 0 511 *:80 *:80 users:((("apache2",pid=7107,fd=4),("apache2",pid=7106,fd=4),("apache2",pid=7105,fd=4),("apache2",pid=7104,fd=4),("apache2",pid=7103,fd=4),("apache2",pid=7100,fd=4)))

(kali@host-3-89)-[~]
$ locate index.html | grep /var/www
/var/www/html/index.html

(kali@host-3-89)-[~]
$ sudo nano /var/www/html/index.html

(kali@host-3-89)-[~]
$

(kali@host-3-89)-[~]
$ sudo systemctl stop apache2
```

Check Tests: Ready

Port of apache2 PASS
apache2 still running PASS

Check Tests: Ready

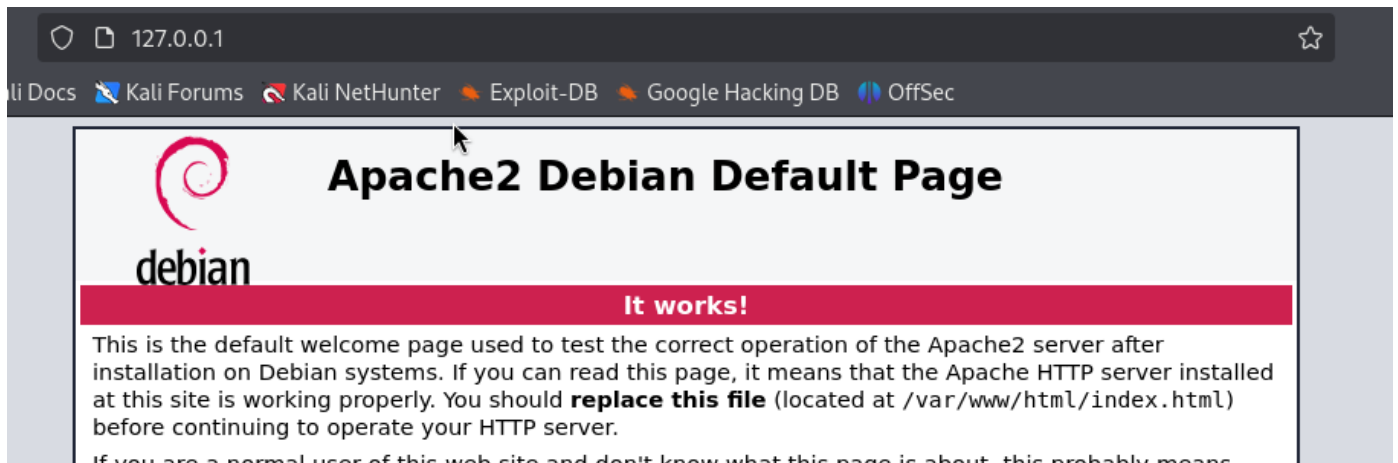
Port of apache2 PASS
apache2 still running PASS

Check Tests: Ready

Port of apache2 PASS
apache2 still running PASS

Use the web browser IN YOUR VIRTUAL MACHINE, browse to 127.0.0.1, and fill in the blank below based on what you can see on the webpage displayed.

The browser can be found in the application menu. Just search for browser.

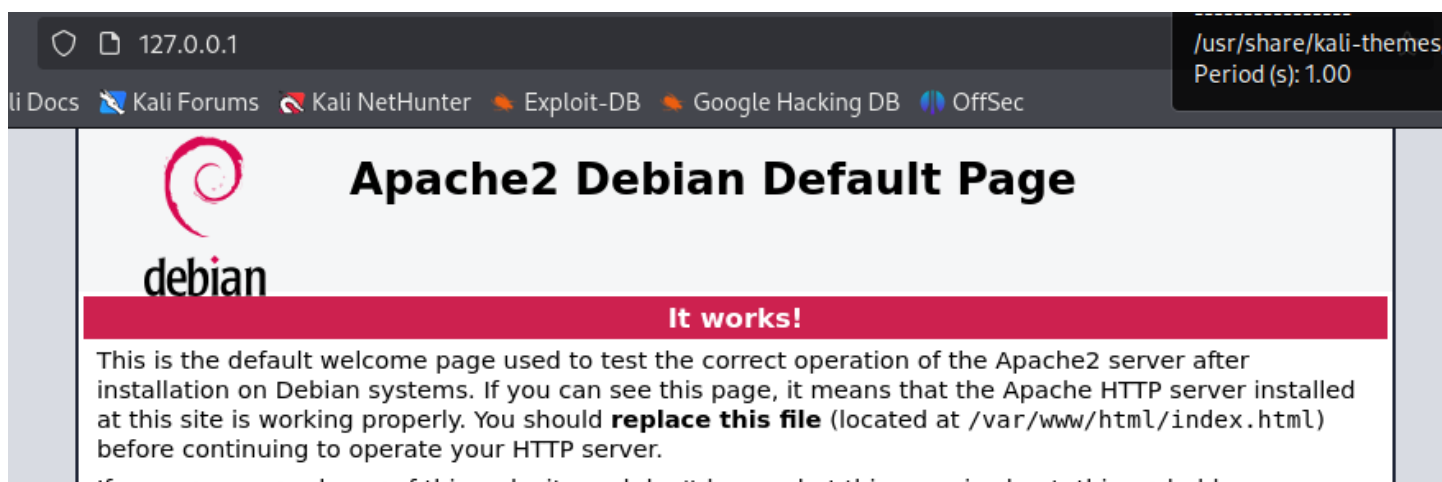


Fill in the blank:

This is the default page used to

Check	Tests: Ready
apache2 is running	PASS
apache2 seems to respond	PASS
blank correct	PASS

Use nano and edit the default webpage.



Use locate to find "index.html", and then pipe and greps to locate an instance below "/var/www".

In the appropriate index.html, change "If you can read this page" to "If you can see this page".

Remember to save! This is a system file, so make sure you do "sudo nano".

Check	Tests: Ready
apache2 seems to respond	PASS file
edited	PASS

Use the "sudo systemctl" command to stop the apache2 service.

Check	Tests: Ready
Stop apache2	PASS

Question 3 Networking

Use the "ip address" command. What is the interface which is connected to the local network? This will be the interface with relates to the 10.x.x.x network.

Device:

Check Tests: Ready

Main network device PASS

Again using "ip address", what is the IP number of this machine in terms of the device identified above.

```
(kali@host-3-89)-[~]
$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:59:00:03:90 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.89/29 brd 10.0.3.95 scope global dynamic noprefixroute eth0
        valid_lft 849333sec preferred_lft 849333sec
    inet6 fe80::20c:59ff:fe00:390/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: labbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc htb state DOWN group default qlen 1000
    link/ether 52:54:00:a6:b7:32 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.254/24 brd 192.168.1.255 scope global labbr0
        valid_lft forever preferred_lft forever
```

IP: 10.0.3.89

Check Tests: Ready

IP number PASS

Question 4 Netcat

Netcat is a command for basic client/server command line configuration.

Use netcat to make a simple ASCII client request to your own SSH server port. Basic netcat configuration puts the server/host IP in parameter 1 and the service port number in parameter 2. Once connected you should see a line of text then the communication seems to hang. At that point hit CTRL-C to terminate the request.

```
(kali@host-3-89)-[~]
$ ncat -l 127.0.0.1 6666

(kali@host-3-89)-[~]
$ netcat 127.0.0.1 6666
```

```
(kali@host-3-89)-[~]
$ netcat 127.0.0.1 22
SSH-2.0-OpenSSH_9.9p1 Debian-3
^C

(kali@host-3-89)-[~]
$ ss -tn | grep 6666
ESTAB 0 127.0.0.1:41268 127.0.0.1:6666
ESTAB 0 127.0.0.1:6666 127.0.0.1:41268

(kali@host-3-89)-[~]
$
```

Looking at the ASCII line you got back from your SSH server, what is the first 7 characters received?

First line:

Check

Tests: Ready

Network working PASS

First PASS

There are different versions of netcat, some with more features than others. In the next few questions I recommend using nmap netcat. This is called ncat. It has some very nice flags which make things easier.

Use ncat for a chat client. Make sure you either have two terminal sessions or two command windows in your virtual machine.

In command window 1, do

```
ncat -l 127.0.0.1 6666
```

In command window 2, connect to this localhost service with netcat on port 6666, and type some messages.

Now have a third command window. Use the ss command and find the established entry for this chat session. What is the local endpoint port number used at the client end of this network connection?

Client port number:

Once passed press CTRL-C in the chat session to quit.

Check

Tests: Ready

Client port number PASS

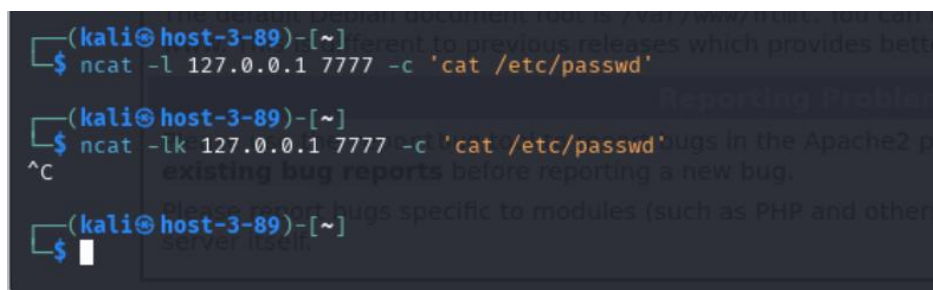
Use

```
-c 'cat /etc/passwd'
```

at the end of the ncat command and run a listening server on port 7777. Try connecting to this, but note that each time you do the end of the connection (when ended with CTRL-C) closes the server too.

Introduce the flag "-k" and restart the ncat listener. Now when you end the client with CTRL-C, you can connect again without problems.

When finished, terminate the listener with a CTRL-C in that window.



Check Tests: Ready

Listener gives /etc/passwd

UNTESTED

Listiner gives /etc/passwd (repeated for continuous) UNTESTED

Use your new knowledge to create a backdoor on port 7890. This should be a listening port in continuous mode which runs /bin/bash when someone connects. Run this and connect to it, and try "ls"...

When finished, terminate the listener with a CTRL-C in that window.

```
(kali@host-3-89) - [~]  
$ netcat 127.0.0.1 7777  
root:x:0:0:root:/root:/usr/bin/zsh  
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin  
bin:x:2:2:bin:/bin:/usr/sbin/nologin  
sys:x:3:3:sys:/dev:/usr/sbin/nologin  
sync:x:4:65534:sync:/bin:/bin/sync  
games:x:5:60:games:/usr/games:/usr/sbin/nologin
```

```
kali:x:1000:1000:Kali Linux,,,:/home/kali:/usr/bin/zsh  
Debian-exim:x:132:135::/var/spool/exim4:/usr/sbin/nologin  
libvirt-qemu:x:64055:994:Libvirt Qemu,,,:/var/lib/libvirt:/usr/sbin/nologin  
^C
```

```
(kali@host-3-89) - [~]  
$ netcat 127.0.0.1 7777  
root:x:0:0:root:/root:/usr/bin/zsh  
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin  
bin:x:2:2:bin:/bin:/usr/sbin/nologin
```

```
kali:x:1000:1000:Kali Linux,,,:/home/kali:/usr/bin/zsh  
Debian-exim:x:132:135::/var/spool/exim4:/usr/sbin/nologin  
libvirt-qemu:x:64055:994:Libvirt Qemu,,,:/var/lib/libvirt:/usr/sbin/nologin  
^C
```

```
(kali@host-3-89) - [~]  
$
```

Check Tests: Ready

Listener gives a shell UNTESTED
Listener gives a shell and continuous UNTESTED

Use your new knowledge to create a server on port 7900. This should be a listening port in continuous mode which runs something which tell you how many files and directories (not including hidden files) can be found in /home/kali. When you are counting you should ignore hidden files.

When finished, terminate the listener with a CTRL-C in that window.

```
(kali@host-3-89) ~  
$ netcat 127.0.0.1 7890  
ls  
Desktop  
Documents  
Downloads  
Music  
Pictures  
Public  
Templates  
Videos  
^C  
(kali@host-3-89) ~  
$ ncat -lp 7890 -k -c /bin/bash  
^C  
(kali@host-3-89) ~  
$
```

```
(kali@host-3-89) ~  
$ ncat -lp 7900 -k -c "find /home/kali -mindepth 1 -maxdepth 1 -not -name '.*' | wc -l"
```

```
(kali@host-3-89) ~  
$ netcat 127.0.0.1 7900  
8
```

`ncat -l -p 7900 -k -c "find /home/kali -mindepth 1 -maxdepth 1 -not -name '.*' | wc -l"`

`ncat``

Network tool that listens or connects to ports.

``-l``

Listen mode (acts like a server).

``-p 7900``

Listen on port **7900**.

``-k``

Keep the server running after one connection (continuous mode).

```
`-c "COMMAND"``
```

Run this command when someone connects.

So everything inside quotes runs when a client connects.

Part 2: The Command Inside Quotes

```
find /home/kali`
```

Search inside `/home/kali`.

```
`-mindepth 1`
```

Do NOT include the starting directory itself.

Without this, `find` would also count:

```
/home/kali
```

We don't want that.

```
`-maxdepth 1`
```

Only look at items directly inside `/home/kali`.

Do NOT go into subfolders.

So it counts:

```
/home/kali/file1
```

```
/home/kali/Documents
```

But NOT:

```
/home/kali/Documents/file2
```

```
`-not -name '.*'`
```

Exclude hidden files.

Hidden files in Linux start with a dot:

```
.bashrc
```

```
.config
```

```
.local
```

This removes them from the count.

```
`| wc -l`
```

The pipe `|` sends the result to:

```
wc -l
```

Which counts how many lines were returned.

Each file = one line

So number of lines = number of files/directories.